FLOWMEDIC ENGINEERING SPECIFICATION

Complete Project Phases Roadmap

Phases 00 through 07 — Technical Architecture, Implementation Checklists & Validation Gates

Repository: FlowMedic	**Stack:** Next.js 15 · Postgres (Neon) · AI Agent
Date: October 2026	**Overall Status:** Phase 00 (Foundation) In Progress

Master Phase Execution Matrix

Phase	Name & Scope	Est. Time	Current Status
Phase 00	Foundation	25 minutes	**COMPLETE**
Phase 01	Real source-to-destination integration	55 minutes	**IN PROGRESS**
Phase 02	Workflow execution and incident capture	40 minutes	NOT STARTED
Phase 03	AI proposal and deterministic validation	50 minutes	NOT STARTED
Phase 04	Human approval and safe replay	45 minutes	NOT STARTED
Phase 05	Recovery Studio workflow canvas	60 minutes	NOT STARTED
Phase 06	Security, hardening, and Vercel deployment	50 minutes	NOT STARTED
Phase 07	Demo rehearsal and contingency	35 minutes	NOT STARTED

Status Legend: **IN PROGRESS**: Active engineering work underway; **NOT STARTED**: Scoped and planned; **COMPLETE**: All exit criteria validated with concrete evidence.

Phase 00 — Foundation

Planned Duration: 25 minutes

COMPLETE

Objective

Create the Next.js App Router + TypeScript skeleton, initial dashboard shell, env variable contract, database migration baseline, and a working health route. Confirm the project builds and runs locally before integration work.

Phase Checklist & Deliverables

[DONE]	Create app using supported current Next.js scaffold and pin package manager/lockfile.
[DONE]	Add Tailwind base, app layout, light navigation shell, and design tokens. shadcn component adoption is deferred to the UI implementation phase.
[DONE]	Create `GET /api/health` with safe config-state output and correlation ID; local response confirmed by the user.
[DONE]	Add `.env.example` containing names only and ignore real environment files.
[DONE]	Add migration directory and initial migration conventions.
[DONE]	Add formatting, lint, typecheck, and build scripts.
[DONE]	Add loading/error and not-connected dashboard states.

Entrance Criteria (Prerequisites)	Exit Criteria (Validation Gates)
• Architecture and decisions reviewed; a dedicated demo GitHub repo and Neon project are planned.	• App starts locally and health route responds. • Build/type checks pass and required env names are documented without values. • No credentials are tracked; initial migration approach is recorded.

Completion Record & Audit

Status	COMPLETE
Actual start/end/duration	Started 2026-10-28; elapsed implementation time not recorded. Completion confirmed 2026-10-28.
Implemented	Next.js App Router project skeleton; light Recovery Studio shell; loading and error states; health Route Handler; `.env.example`; migration conventions; formatting, lint, typecheck, and build scripts. See repository files and **UI design**.
Deviations and decisions	Webpack is selected in the dev/build scripts after the Turbopack CSS pipeline failed in this environment; the Webpack production build passed. shadcn components are deferred to Phase 05, where the full UI is implemented.

Verification	`npm run format`, `npm run lint`, `npm run typecheck`, and `npm run build` passed in the implementation session. The user subsequently confirmed the local app and `/api/health` check.
Evidence	Build output included `/`, `/_not-found`, and `/api/health`; the user confirmed the local health check in chat on 2026-10-28. Code is present in `src/app/`, environment contract in `.env.example`, and migration guidance in `db/migrations/README.md`.
Blockers / owner / next action	None for Phase 00. Proceed to Phase 01.
Known gaps carried forward	No GitHub, Postgres connection, LLM, authentication, or live run orchestration; no initial database schema; health HTTP response not confirmed. These are tracked for later phases or the remaining Phase 00 check.

Phase 01 — Real source-to-destination integration

Planned Duration: 55 minutes

IN PROGRESS

Objective

Connect a least-privilege GitHub token and managed Postgres; fetch a bounded page of real issues, normalize them, and upsert them with stable keys.

Phase Checklist & Deliverables

[TODO]	Provision a dedicated demo repository/database and configure local/Vercel secrets. Use a GitHub fine-grained token with Issues: read permission for only the demo repository.
[DONE]	Add versioned migration for `github_issues` and run/step tables; `issue_title` is the intentionally driftable target field. Migration still needs to be applied to Neon.
[DONE]	Implement server-only GitHub REST client with an 8-second request timeout, maximum two pages of 50 items, and safe authorization/rate-limit errors.
[DONE]	Exclude pull requests; normalize stable issue fields and cap issue body length at 4,000 characters.
[DONE]	Implement parameterized Neon Postgres upsert with unique source/repository/external ID.
[DONE]	Add token-protected manual `POST /api/runs`, durable run/step checkpoints, idempotency key handling, and post-write count verification.
[DONE]	Add Recovery Studio control for configured service flags, manual sync, safe request status, and actual sync counts after completion.

Entrance Criteria (Prerequisites)	Exit Criteria (Validation Gates)
• Phase 00 complete; demo GitHub/Neon services and credentials are available; migration is applied.	• Manual sync fetches actual issues and verifies rows in Neon. • Repeating same sync updates existing records rather than duplicating. • Errors are surfaced safely and run state persists.

Completion Record & Audit

Status	IN PROGRESS
Actual start/end/duration	Started 2026-10-28; elapsed time not recorded.
Implemented	Server-only GitHub client in `src/lib/github.ts`; Neon persistence in `src/lib/database.ts`; protected manual sync endpoint in `src/app/api/runs/route.ts`; Phase 01 schema in `db/migrations/0001_phase_01_source_sync.sql`; phase setup notes in `db/migrations/README.md`.

Deviations and decisions	Manual endpoint uses a dedicated `FLOWMEDIC_RUN_TOKEN` bearer secret until full operator-session authentication is implemented. Each run is bounded to at most 100 GitHub issues; idempotency applies to the caller-provided run key, while issue upserts are independently unique.
Verification	Not run in this implementation pass, per the user's request to keep checks lightweight. No GitHub or Neon credentials were supplied, so no external sync or deployed row verification is claimed.
Evidence	Code and migration listed above. GitHub fine-grained tokens need Issues: read permission to list repository issues: https://docs.github.com/en/rest/issues/issues. Next.js Route Handlers support the `POST` endpoint pattern: https://nextjs.org/docs/app/getting-started/route-handlers.
Blockers / owner / next action	Owner: project operator. Provision/configure demo GitHub and Neon, apply migration, then perform one manual sync and confirm repeat sync does not duplicate rows.
Known gaps carried forward	Migration is not yet applied; no GitHub/Neon credentials were available; authentication is a shared operator bearer token rather than a user session; no integration test or live-provider check has been run. UI reports environment configuration, not a verified external connection.

Phase 02 — Workflow execution and incident capture

Planned Duration: 40 minutes

NOT STARTED

Objective

Persist workflow runs, steps, safe error evidence, and incidents. Demonstrate a genuine PostgreSQL schema-drift failure only in the dedicated demo database.

Phase Checklist & Deliverables

[TODO]	Add migrations for `workflow_steps`, `incidents`, and audit events.
[TODO]	Implement state machine and guarded transitions with version checks.
[TODO]	Add run/step correlation IDs, duration, status, error class, and safe summaries.
[TODO]	Provide explicit demo-only schema rename procedure (migration/manual action) from `issue_title` to `title`.
[TODO]	Trigger a fresh sync and capture actual undefined-column SQLSTATE/error.
[TODO]	Persist sanitized payload snapshot and incident fingerprint.
[TODO]	Add basic incident list/detail read APIs.

Entrance Criteria (Prerequisites)	Exit Criteria (Validation Gates)
• Phase 01 complete; normal sync verified in a disposable database.	• Controlled rename causes a real DB failure. • Incident and failed step persist with safe evidence; no secrets/raw headers logged. • A retry does not create duplicate incidents for the same failing operation.

Completion Record & Audit

Status	NOT STARTED
Actual start/end/duration	—
Implemented	—
Deviations and decisions	—
Verification	—
Evidence	—

Blockers / owner / next action	—
Known gaps carried forward	—

Phase 03 — AI proposal and deterministic validation

Planned Duration: 50 minutes

NOT STARTED

Objective

Generate a schema-constrained repair proposal from minimal incident evidence, then validate it with code-owned allowlists and non-mutating checks.

Phase Checklist & Deliverables

[TODO]	Define versioned JSON schema for diagnosis and source/target field mappings.
[TODO]	Implement LLM server client with timeout, token budget, safe retry policy, and provider metadata.
[TODO]	Redact sensitive content and bound prompt evidence.
[TODO]	Treat issue text/errors as untrusted; explicitly instruct model not to follow embedded instructions.
[TODO]	Implement deterministic allowlist/type/required-field/duplicate-target validator.
[TODO]	Reject arbitrary SQL, unknown identifiers, DDL, expressions, and incompatible types.
[TODO]	Implement dry-run that verifies columns and payload shape without mutation.
[TODO]	Persist proposal, prompt version, validation outcome, and audit events.
[TODO]	Demonstrate known `issue_title → title` accepted and unsafe mapping rejected.

Entrance Criteria (Prerequisites)	Exit Criteria (Validation Gates)
• Phase 02 complete with persisted reproducible incident evidence; LLM key configured.	• Valid structured proposal shown with validation result and evidence. • Known case is accepted; adversarial/unknown target is rejected deterministically. • No proposal can write or mutate schema in this phase.

Completion Record & Audit

Status	NOT STARTED
Actual start/end/duration	—
Implemented	—
Deviations and decisions	—
Verification	—
Evidence	—

Blockers / owner / next action	—
Known gaps carried forward	—

Phase 04 — Human approval and safe replay

Planned Duration: 45 minutes

NOT STARTED

Objective

Require an authenticated human to approve the exact validated proposal, then replay once idempotently and verify the destination.

Phase Checklist & Deliverables

[TODO]	Add proposal approval/rejection records and immutable proposal hash.
[TODO]	Bind approval to actor, proposal hash, incident version, and validation result.
[TODO]	Enforce authenticated action, current state, and stale-version rejection.
[TODO]	Generate SQL from code-owned validated identifiers; parameterize issue values.
[TODO]	Use original run/issue idempotency key and unique destination key on replay.
[TODO]	Bound replay attempt count; reject duplicate or concurrent replay.
[TODO]	Read back and verify destination row before marking recovered.
[TODO]	Record approval, replay attempt, and terminal outcome in append-only audit.

Entrance Criteria (Prerequisites)	Exit Criteria (Validation Gates)
• Phase 03 complete; dashboard authentication and proposal validation available.	• Replay is impossible before approval and impossible for an altered/stale proposal. • Approved mapping succeeds and verifies; repeating replay creates no duplicate. • Rejection and replay failure have clear persisted states.

Completion Record & Audit

Status	NOT STARTED
Actual start/end/duration	—
Implemented	—
Deviations and decisions	—
Verification	—
Evidence	—
Blockers / owner / next action	—

Known gaps carried forward	—

Phase 05 — Recovery Studio workflow canvas

Planned Duration: 60 minutes
Reference: [`docs/UI_DESIGN.md`](../UI_DESIGN.md)

NOT STARTED

Objective

Build a polished operator workspace inspired by node-based workflow canvases: show each real step progressing through the sync and recovery path, with a live execution rail and contextual inspector. Make incident evidence, safe approval, and recovery understandable at a glance.

Phase Checklist & Deliverables

[TODO]	Build responsive dark Recovery Studio shell with accessible contrast, quiet canvas grid, toolbar, and status labels.
[TODO]	Render React Flow topology for GitHub → normalize/map → Postgres and a visible incident → diagnose → validate → approval → replay recovery path.
[TODO]	Display real active step, elapsed time, finished/failed/queued counts, and factual event sequence; never estimate progress by time.
[TODO]	Persist step-start and step-terminal events, and advance one server-authorized bounded step per request.
[TODO]	Drive node/edge status from persisted run/step status, not timer-only animation.
[TODO]	Add run history, incident detail, sanitized payload/error, diagnosis, and timeline.
[TODO]	Add proposal mapping diff, validation evidence, approve/reject confirmation, replay status.
[TODO]	Poll run/incident endpoints with bounded backoff and stop on terminal state.
[TODO]	Include loading, empty, stale, failure, and permission-denied states.
[TODO]	Ensure keyboard operation and clear destructive-action labeling.

Entrance Criteria (Prerequisites)	Exit Criteria (Validation Gates)
• Phases 01–04 provide stable APIs and seeded real workflow states.	• Dashboard communicates success, live failure, approval wait, and recovery from real server state. • Operator can trigger sync, inspect incident, approve/reject, and follow replay. • Demo UI never implies an unperformed repair succeeded.

Completion Record & Audit

Status	NOT STARTED
Actual start/end/duration	—
Implemented	—
Deviations and decisions	—
Verification	—
Evidence	—
Blockers / owner / next action	—
Known gaps carried forward	—

Phase 06 — Security, hardening, and Vercel deployment

Planned Duration: 50 minutes

NOT STARTED

Objective

Deploy one Vercel project, validate real environment configuration, secure all integration boundaries, and keep all requests bounded and observable.

Phase Checklist & Deliverables

[TODO]	Review every route for auth, authorization, CSRF/origin, validation, size limits, and rate limits.
[TODO]	Verify secrets are server-only, absent from client bundle, repo, logs, screenshots, and errors.
[TODO]	Verify GitHub webhook HMAC and delivery dedupe if webhook included; protect cron with secret if cron included.
[TODO]	Verify DB least privilege, pooler URL, region placement, migration state, and connection pressure.
[TODO]	Set service timeouts and function `maxDuration` appropriate to actual Vercel plan.
[TODO]	Ensure work persists before response/timeout and can resume; no process-local locks/background assumptions.
[TODO]	Configure Vercel env variables separately for preview/production and deploy preview.
[TODO]	Review logs, errors, URL/auth redirects, and provider rate limits.
[TODO]	Run security and end-to-end checks against disposable services.

Entrance Criteria (Prerequisites)	Exit Criteria (Validation Gates)
• Core demo flow works locally; credentials and Vercel project available.	• Preview deployment completes and critical integration flow runs against demo services. • Actual plan limits and route durations are checked and recorded. • No secret exposure; failure states are persisted and visible.

Completion Record & Audit

Status	NOT STARTED
Actual start/end/duration	—
Implemented	—
Deviations and decisions	—

Verification	—
Evidence	—
Blockers / owner / next action	—
Known gaps carried forward	—

Phase 07 — Demo rehearsal and contingency

Planned Duration: 35 minutes

NOT STARTED

Objective

Make the end-to-end story repeatable, evidence-backed, and resilient to external service/deployment issues.

Phase Checklist & Deliverables

[TODO]	Prepare operator runbook: connect, sync, verify row, create controlled drift, trigger, inspect, approve, verify replay.
[TODO]	Rehearse twice from clean demo state and measure timing.
[TODO]	Capture proof of real GitHub issue, actual DB error, approval event, and verified recovered row.
[TODO]	Prepare safe reset procedure scoped only to demo tables/schema.
[TODO]	Prepare fallback for LLM outage (clearly labeled deterministic proposal) and deployment outage (local run disclosed).
[TODO]	Verify no secret appears in screen recording or presentation.
[TODO]	Finalize known limitations and phase/changelog evidence.
[TODO]	Confirm demo does not imply autonomous unapproved action or general-purpose code repair.

Entrance Criteria (Prerequisites)	Exit Criteria (Validation Gates)
• Phase 06 deployed preview or documented local fallback; core flow complete.	• Two successful rehearsals or a documented, understood blocker and fallback. • Runbook, evidence, fallback, cleanup, and limitations are reviewable. • Phase log and changelog are current and consistent.

Completion Record & Audit

Status	NOT STARTED
Actual start/end/duration	—
Implemented	—
Deviations and decisions	—
Verification	—
Evidence	—

Blockers / owner / next action	—
Known gaps carried forward	—